



Research Intake 2026

Day 20

Fine-Tuning Language Models

A Beginner's Guide for Students

From foundational concepts to research-level understanding

Gudsky Research Foundation

Table of Contents

1	Transfer Learning in NLP	4
1.1	The ImageNet Moment and Its NLP Analogue	4
1.2	Why Fine-Tuning Works: The Representation Reuse View.....	4
1.3	Cost Comparison: From-Scratch vs. Fine-Tuning	4
1.4	Sequential Transfer vs. Multi-Task Transfer	5
2	Full Fine-Tuning vs. Feature Extraction	7
2.1	Feature Extraction (Frozen Backbone)	7
2.2	Full Fine-Tuning (Updating All Weights)	7
2.3	Partial Fine-Tuning and Gradual Unfreezing.....	7
2.4	A Practical Decision Framework	8
3	Hugging Face Trainer API.....	10
3.1	TrainingArguments and Data Collators	10
	TrainingArguments	10
	Data Collators	10
3.2	Metrics and Evaluation Callbacks.....	11
3.3	Subclassing Trainer for Custom Loss Functions	12
3.4	Troubleshooting Common Trainer Issues.....	13
3.5	Distributed Training Considerations.....	13
4	Learning Rate Scheduling	15
4.1	Warm-Up and Linear Decay	15
4.2	Cosine Annealing and the One-Cycle Policy.....	15
	Cosine Annealing.....	16
	The One-Cycle Policy.....	16
4.3	Differential Learning Rates Across Layers.....	17
5	Catastrophic Forgetting and Mitigation	18
5.1	Why Catastrophic Forgetting Happens	18
5.2	Mitigation Strategies.....	18
	Small Learning Rates and Limited Epochs.....	18
	Layer Freezing	18
	Elastic Weight Consolidation (EWC)	19
	Rehearsal and Replay.....	19
	Mixout.....	19
5.3	Measuring Forgetting: Evaluation Protocols	20
6	Parameter-Efficient Fine-Tuning (PEFT)	21
6.1	Adapter Layers.....	21

6.2	LoRA: Low-Rank Adaptation.....	22
6.3	Prefix Tuning and Prompt Tuning	22
	Prefix Tuning	23
	Prompt Tuning	23
6.4	Comparing PEFT Methods	23
6.5	IA ³ : A Lighter-Weight Alternative	24
6.6	Choosing a PEFT Method.....	24
6.7	Worked Example: LoRA Fine-Tuning with Hugging Face PEFT.....	24
7	Domain Adaptation vs. Task Adaptation.....	26
7.1	Domain Adaptation.....	26
7.2	Task Adaptation.....	26
7.3	Combining Both: DAPT + TAPT	26
7.4	Worked Example: Adapting a General Model for a Specialised Research Domain.....	27
8	Research: Instruction Tuning and RLHF	29
8.1	Instruction Tuning.....	29
8.2	Reinforcement Learning from Human Feedback (RLHF).....	29
8.3	Direct Preference Optimisation (DPO): A Simpler Alternative.....	30
8.4	The Alignment Tax and Open Research Questions	31
8.5	Summary: From Task Fine-Tuning to Alignment	31

1 Transfer Learning in NLP

Transfer learning — the practice of reusing a model trained on one task as the starting point for a model on a related task — had already proven transformative in computer vision. Krizhevsky et al.'s 2012 ImageNet-trained CNN, and the wave of vision models that followed (VGG, ResNet, Inception), demonstrated that features learned on a large labelled dataset (1.2 million images, 1,000 categories) generalised remarkably well to entirely different visual tasks: medical image classification, satellite imagery analysis, even art style recognition, often with only a few thousand task-specific labelled examples.

NLP lagged behind this paradigm for years, largely because no equivalent of ImageNet existed for language — there was no single, massive, labelled dataset that could anchor a universal pre-training signal. The breakthrough came not from finding a labelled dataset at ImageNet scale, but from realising that labels were unnecessary: self-supervised objectives (next-word prediction, masked-word prediction) could extract a comparably rich training signal from raw, unlabelled text, which exists in effectively unlimited quantity. ELMo, GPT, and BERT each represent successive refinements of this insight, with BERT (covered in Day 19) bringing the full transfer-learning paradigm to NLP at scale.

1.1 The ImageNet Moment and Its NLP Analogue

The parallel between vision and NLP transfer learning is instructive precisely because of where it breaks down. In vision, pre-training and fine-tuning operate on the same modality and similar object statistics — edges, textures, and shapes recur across natural images regardless of domain. In NLP, pre-training and fine-tuning can differ far more dramatically: a model pre-trained on Wikipedia prose must transfer to legal contracts, casual tweets, biomedical abstracts, or customer support transcripts — domains with wildly different vocabulary, syntax, and discourse conventions. This makes the fact that NLP transfer learning works at all, and works well, a more remarkable empirical finding than its vision counterpart.

The practical consequence for the Gudskey team's research workflow is direct: a single pre-trained checkpoint (BERT, RoBERTa, a LLaMA-family model, etc.) can serve as the starting point for nearly any downstream NLP task, and the cost of obtaining a strong task-specific model is now overwhelmingly the cost of fine-tuning, not pre-training. Pre-training is performed once, by an organisation with the compute budget to do so (Google, Meta, OpenAI); fine-tuning is performed many times, by researchers and engineers adapting that single investment to thousands of downstream applications.

1.2 Why Fine-Tuning Works: The Representation Reuse View

The theoretical justification for why fine-tuning a pre-trained model outperforms training from scratch rests on the idea of representation reuse. During pre-training, the model's parameters converge toward a region of weight space that encodes general-purpose linguistic structure — the syntactic and semantic hierarchies discussed in Day 19's probing analysis. Fine-tuning does not need to discover this structure again; it only needs to make small, task-specific adjustments to representations that are already well-organised.

This can be understood through the lens of the loss landscape: training from scratch starts at a random point in an extremely high-dimensional, non-convex loss surface and must navigate toward a good solution using only the gradient signal from a small labelled dataset — a search that is easy to get lost in. Fine-tuning instead starts from a point that pre-training has already identified as a good general-purpose solution, and only needs to traverse a short distance to specialise it for the task at hand. Empirically, this means fine-tuning requires dramatically fewer gradient steps, less labelled data, and is less sensitive to weight initialisation than training from scratch.

1.3 Cost Comparison: From-Scratch vs. Fine-Tuning

The practical cost savings from transfer learning are large enough to have reshaped how NLP research and engineering teams allocate their resources. The table below summarises the typical resource requirements for building a strong sentiment classifier from scratch versus fine-tuning a pre-trained model.

Resource	Training From Scratch	Fine-Tuning Pre-Trained Model
Labelled examples needed	100,000+	1,000 – 10,000
Training time (single GPU)	Days to weeks	Minutes to hours
Compute cost	High (full architecture search + training)	Low (few epochs on small data)
Risk of overfitting	High with limited data	Lower — representations already general
Required ML expertise	Significant (architecture design)	Moderate (hyperparameter tuning)

Fine-Tuning BERT in 30 Minutes

A standard fine-tuning run (3 epochs, batch size 32, learning rate $2e-5$) on the SST-2 sentiment dataset (11,000 examples) takes approximately 30 minutes on a single V100 GPU and achieves 93% accuracy. The same BERT model, fine-tuned on the SQuAD reading comprehension dataset, achieves human-parity performance on paragraph-level question answering — demonstrating the universality of pre-trained representations across tasks as different as sentiment classification and span-extraction QA.

1.4 Sequential Transfer vs. Multi-Task Transfer

Two distinct transfer learning patterns recur across research and applied NLP workflows, and they carry different practical implications for how a model's knowledge accumulates or erodes over time. Sequential transfer refers to the standard pattern discussed throughout this section: a single pre-trained checkpoint is fine-tuned once for a single downstream task, producing a model specialised for that task alone. If a second, different task later needs to be supported, a fresh copy of the original pre-trained checkpoint is typically fine-tuned independently, rather than continuing to fine-tune the already-specialised first model.

Multi-task transfer, by contrast, fine-tunes a single model simultaneously across several tasks, typically by sampling mini-batches from each task's dataset in an interleaved fashion and using a shared backbone with either a single shared output head (when tasks share a label space) or multiple task-specific heads attached to the same backbone (when they do not). Multi-task fine-tuning often produces representations that generalise better than single-task fine-tuning, because the model is forced to learn features useful across multiple objectives simultaneously rather than overfitting narrowly to one task's idiosyncrasies — a phenomenon sometimes described as task regularisation.

The choice between these patterns interacts directly with the catastrophic forgetting concerns explored in Section 5: naive sequential fine-tuning across multiple tasks, one after another using the same model instance, is exactly the scenario in which forgetting of earlier tasks is most severe. Multi-task fine-tuning avoids this failure mode by construction, since all tasks are learned concurrently rather than sequentially, but it requires having access to all task datasets at once — which is not always feasible in a Gudskey research pipeline where new tasks or datasets may only become available later in a project's timeline.

Pattern	Data Availability Requirement	Forgetting Risk	Typical Representation Quality
Sequential Transfer (single task)	One task dataset at a time	Low (only one task ever learned)	Good — specialised for that task
Sequential Transfer (multiple tasks, same model)	Tasks arrive at different times	High — later tasks overwrite earlier ones	Degrades for earlier tasks over time
Multi-Task Transfer	All task datasets available together	Low — tasks learned concurrently	Often best — shared regularisation effect

2 Full Fine-Tuning vs. Feature Extraction

Once a pre-trained model is selected, the researcher must choose how to adapt it to the downstream task. There are two classical strategies — full fine-tuning and feature extraction — along with intermediate approaches (gradual unfreezing, partial fine-tuning) that interpolate between them. The right choice depends on dataset size, compute budget, task similarity to pre-training, and deployment constraints.

2.1 Feature Extraction (Frozen Backbone)

In feature extraction, the pre-trained model's parameters are frozen entirely — no gradients flow through the backbone during training. The model is used purely as a fixed function that maps raw text to a dense vector representation (typically the [CLS] hidden state, or a pooled average of token representations). A small task-specific head — usually a single linear layer or a shallow MLP — is trained on top of these frozen features.

This approach has several practical advantages. Because the backbone is frozen, its representations for a given input can be pre-computed once and cached, dramatically speeding up training when the same backbone is reused across many experiments or tasks. Training is fast and memory-efficient, since gradients and optimiser states are only needed for the small head, not the full model — this makes feature extraction viable even on hardware too limited to fine-tune the full model. It also reduces the risk of catastrophic forgetting (Section 5), since the pre-trained representations are never modified.

The trade-off is a ceiling on performance: because the backbone's representations are fixed, the model cannot adapt its internal features to better suit the downstream task. For tasks that are similar to the pre-training objective (general sentence classification), feature extraction performs reasonably well. For tasks that require representations not well-captured by the frozen backbone (highly specialised domains, structurally different tasks), feature extraction underperforms full fine-tuning by a meaningful margin.

2.2 Full Fine-Tuning (Updating All Weights)

In full fine-tuning, every parameter in the pre-trained model — including all transformer layers, attention weights, and embeddings — is updated during training, typically alongside a newly initialised task-specific head. Gradients flow through the entire network, allowing every layer's representations to be adjusted in service of the downstream task.

Full fine-tuning consistently achieves the strongest task performance among the strategies discussed in this section, because it grants the model maximum flexibility to specialise its representations. This is the strategy used in the original BERT paper's downstream task evaluations (Day 19, Section 3) and remains the default choice when compute and labelled data are not severely constrained.

The costs are correspondingly higher: full fine-tuning requires storing gradients and optimiser states (e.g., Adam's first and second moment estimates) for every parameter in the model, multiplying memory requirements by a factor of 3–4× relative to inference-only memory use. For a 7-billion-parameter model, this can mean the difference between fitting on a single consumer GPU and requiring a multi-GPU cluster — a key motivation for the parameter-efficient methods covered in Section 6.

2.3 Partial Fine-Tuning and Gradual Unfreezing

Between the two extremes lies partial fine-tuning, in which only a subset of the model's layers are updated — commonly the upper layers closest to the output, while lower layers remain frozen. This is motivated by the probing-classifier findings discussed in Day 19, Section 9: lower transformer layers tend to encode general syntactic structure useful across nearly all tasks, while upper layers encode more task-relevant

semantic and contextual information. Freezing the lower layers preserves general-purpose representations while still allowing task-specific specialisation in the upper layers.

Gradual unfreezing (introduced in the context of ULMFiT, Howard & Ruder, 2018) is a scheduling strategy for partial fine-tuning: training begins with only the final layer (or task head) unfrozen, and additional layers are progressively unfrozen — typically one per epoch, starting from the top — over the course of training. This staged approach reduces the risk of catastrophic forgetting in early training, when gradients from a randomly initialised task head can be large and disruptive if allowed to propagate through the entire pre-trained network immediately.

Strategy	Parameters Updated	Compute Cost	Typical Performance	Best For
Feature Extraction	Task head only	Lowest	Good	Small datasets, limited compute, rapid prototyping
Partial Fine-Tuning	Upper layers + head	Moderate	Very Good	Moderate datasets, memory-constrained settings
Full Fine-Tuning	All parameters	Highest	Best	Large datasets, maximum accuracy requirements
Gradual Unfreezing	All, staged over time	High	Very Good — Best	Avoiding early-training instability

2.4 A Practical Decision Framework

In practice, the choice between feature extraction, partial fine-tuning, and full fine-tuning is rarely made from first principles alone — it is shaped by the concrete constraints of a given research project. The following decision framework, distilled from common practice across the NLP research community, offers a starting heuristic for the Gudsky team's experimentation pipeline.

When the labelled dataset is very small (under roughly 1,000 examples), feature extraction is often the safer choice: full fine-tuning on so little data carries a substantial risk of overfitting, since the number of trainable parameters in the task head is small relative to the number of training examples, whereas full fine-tuning would expose hundreds of millions of parameters to a dataset far too small to constrain them meaningfully. When the labelled dataset is moderate (roughly 1,000–10,000 examples), partial fine-tuning or full fine-tuning with a conservative learning rate and few epochs typically outperforms feature extraction, since there is now sufficient signal to productively adjust at least the upper layers. When the labelled dataset is large (tens of thousands of examples or more), full fine-tuning is almost always preferable, provided the compute budget allows it, since the larger dataset both justifies and constrains the additional parameter flexibility.

A second consideration, orthogonal to dataset size, is downstream deployment: if many different task-specific models must be served simultaneously from the same base architecture, feature extraction (or the PEFT methods covered in Section 6) dramatically reduces storage and serving costs, since only the small task head — or small adapter module — needs to be stored and loaded per task, rather than a full copy of the fine-tuned backbone for every task.



Quantifying the Memory Gap: Full Fine-Tuning vs. Feature Extraction

For a BERT-Large model (340M parameters) trained with the Adam optimiser:

Full fine-tuning memory: ~ 1.4 GB (fp32 weights) + ~ 1.4 GB (gradients) + ~ 2.8 GB (Adam moments) ≈ 5.6 GB,

before accounting for activations — easily exceeding 16 GB total with a reasonable batch size and sequence length.

Feature extraction memory: ~ 1.4 GB (frozen weights, no gradients needed) + negligible head parameters $\approx$ well under 2 GB total, since no optimiser state is needed for the frozen backbone at all.

This roughly 3–4 $\times$ memory gap is precisely why feature extraction remains attractive on constrained hardware,

even though full fine-tuning typically yields the strongest downstream accuracy when memory allows for it.

3 Hugging Face Trainer API

The Hugging Face `transformers` library provides a high-level `Trainer` class that abstracts away the boilerplate of the standard PyTorch training loop — gradient accumulation, mixed-precision training, distributed training, checkpointing, logging, and evaluation — while remaining flexible enough to support the customisation that research workflows typically require. For the Gudsky team's experimentation pipeline, the `Trainer` API is the default entry point for fine-tuning any Hugging Face model, from BERT-Base sentiment classifiers to LoRA-adapted LLaMA models.

The `Trainer` is configured through three primary components: a `TrainingArguments` object specifying all training hyperparameters and logging behaviour, a data collator that batches and pads raw examples into model-ready tensors, and an optional set of callbacks and metric functions for evaluation during training. Understanding how these components interact is essential for diagnosing training issues and customising the pipeline for non-standard tasks.

3.1 TrainingArguments and Data Collators

TrainingArguments

`TrainingArguments` is a configuration dataclass that centralises every hyperparameter governing the training run: learning rate, batch size, number of epochs, weight decay, warmup steps, evaluation strategy, checkpoint saving frequency, mixed-precision settings, and dozens of other options. Centralising these settings in a single object — rather than scattering them through a manual training loop — makes experiments reproducible and easy to compare across runs.

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=64,
    learning_rate=2e-5,
    weight_decay=0.01,
    warmup_ratio=0.1,          # 10% of steps for LR warmup
    lr_scheduler_type='linear',
    eval_strategy='epoch',     # evaluate after every epoch
    save_strategy='epoch',
    load_best_model_at_end=True,
    metric_for_best_model='f1',
    fp16=True,                # mixed-precision training
    logging_dir='./logs',
    logging_steps=50,
    report_to='tensorboard',
)
```

Data Collators

A data collator is a function (or callable class) that takes a list of individual examples from the dataset and combines them into a single batched, padded tensor suitable for the model's forward pass. This step is

necessary because individual text sequences vary in length, while tensors in a batch must have uniform shape.

The most commonly used collator, `DataCollatorWithPadding`, dynamically pads each batch to the length of the longest sequence in that specific batch — rather than padding every example in the dataset to a single global maximum length. This is more computationally efficient: batches consisting of short sequences avoid unnecessary padding tokens, reducing wasted compute on positions that contribute nothing to the loss. For masked language modelling fine-tuning, `DataCollatorForLanguageModeling` additionally applies the MLM masking procedure (Day 19, Section 2.1) dynamically at batch-construction time.

```
from transformers import AutoTokenizer, DataCollatorWithPadding

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
data_collator = DataCollatorWithPadding(tokenizer=tokenizer, padding='longest')

# Custom collator example: adding label smoothing or custom padding logic
def custom_collate_fn(batch):
    input_ids = [item['input_ids'] for item in batch]
    labels = [item['label'] for item in batch]
    encoded = tokenizer.pad({'input_ids': input_ids}, return_tensors='pt')
    encoded['labels'] = torch.tensor(labels)
    return encoded
```

3.2 Metrics and Evaluation Callbacks

The `Trainer` evaluates the model periodically during training (controlled by `eval_strategy`) by running the evaluation dataset through the model and computing a user-defined `compute_metrics` function on the resulting predictions. This function receives an `EvalPrediction` object containing the raw model logits and the true labels, and must return a dictionary of metric names to values.

```
import numpy as np
from sklearn.metrics import accuracy_score, f1_score, precision_recall_fscore_support

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average='weighted'
    )
    accuracy = accuracy_score(labels, predictions)
    return {
        'accuracy': accuracy,
        'f1': f1,
        'precision': precision,
        'recall': recall,
    }

trainer = Trainer(
```

```

model=model,
args=training_args,
train_dataset=train_dataset,
eval_dataset=eval_dataset,
data_collator=data_collator,
compute_metrics=compute_metrics,
)
trainer.train()

```

Callbacks extend the `Trainer`'s behaviour at specific points in the training loop — at the start or end of training, at the end of every epoch, after every evaluation, and so on — without needing to modify the `Trainer`'s internal code. The most commonly used built-in callback is `EarlyStoppingCallback`, which halts training automatically once a monitored metric (e.g., validation F1) stops improving for a specified number of evaluation rounds, preventing wasted compute on overfitting.

```

from transformers import EarlyStoppingCallback

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=3)],
)

```

Custom callbacks can be written by subclassing `TrainerCallback` and overriding hooks such as `on\_epoch\_end`, `on\_evaluate`, or `on\_log` — useful for logging to external experiment-tracking systems (Weights & Biases, MLflow), implementing custom learning rate adjustments, or triggering alerts when a metric crosses a threshold relevant to a research milestone.

3.3 Subclassing Trainer for Custom Loss Functions

The default `Trainer` assumes a standard supervised loss computed directly from the model's output and the provided labels — appropriate for most classification, regression, and span-extraction tasks. For research workflows that require a non-standard loss — for example, a weighted cross-entropy loss to handle severe class imbalance, a custom contrastive loss, or the combined MLM-plus-distillation loss discussed in Day 19's treatment of DistilBERT — the `Trainer` class can be subclassed and its `compute\_loss` method overridden, while retaining all the surrounding infrastructure (checkpointing, logging, mixed-precision, distributed training) unchanged.

```

from transformers import Trainer
import torch.nn as nn

class WeightedLossTrainer(Trainer):
    def __init__(self, *args, class_weights=None, **kwargs):
        super().__init__(*args, **kwargs)

```

```

self.class_weights = class_weights

def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
    labels = inputs.pop('labels')
    outputs = model(**inputs)
    logits = outputs.logits
    loss_fn = nn.CrossEntropyLoss(weight=self.class_weights)
    loss = loss_fn(logits.view(-1, model.config.num_labels), labels.view(-1))
    return (loss, outputs) if return_outputs else loss

trainer = WeightedLossTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    class_weights=torch.tensor([1.0, 3.5]), # up-weight minority class
)

```

3.4 Troubleshooting Common Trainer Issues

A handful of recurring issues account for the majority of problems researchers encounter when first using the `Trainer` API, and a brief troubleshooting reference is worth including here given its practical value to the Gudskey team's day-to-day fine-tuning work.

- **Out-of-memory errors:** Reduce `per\_device\_train\_batch\_size` and compensate with `gradient\_accumulation\_steps` to maintain the same effective batch size; enable `fp16` or `bf16` mixed-precision training; or apply one of the PEFT methods from Section 6 to reduce optimiser state memory.
- **Loss not decreasing:** Verify the learning rate is not too high (Section 4) or too low; confirm the data collator is producing correctly shaped and labelled batches by inspecting a single batch manually before training; check that the task head's output dimension matches the number of classes in the dataset.
- **Evaluation metric not improving despite decreasing training loss:** This is a classic overfitting signature — reduce the number of epochs, apply weight decay, or switch from full fine-tuning to a more conservative strategy (Section 2.3) given the dataset size.
- **Training is unexpectedly slow:** Confirm the data collator is using dynamic (not maximum-length) padding; confirm the model and data are correctly placed on GPU rather than silently falling back to CPU; profile data loading versus forward/backward pass time to identify the bottleneck.

3.5 Distributed Training Considerations

As model and dataset sizes grow, fine-tuning workflows increasingly need to scale beyond a single GPU. The `Trainer` API integrates transparently with several distributed training backends, requiring minimal changes to the training script itself — the same code used for single-GPU training will automatically scale to multiple GPUs or multiple machines when launched appropriately.

Distributed Data Parallel (DDP), the most common distributed training strategy for fine-tuning, replicates the full model on every GPU and splits each training batch across them; gradients computed on each GPU are synchronised (averaged) after every backward pass, so that every model replica stays in sync. DDP is well-suited to fine-tuning scenarios where the model fits comfortably in a single GPU's memory and the goal of distribution is purely to increase effective batch size and training throughput.

For models too large to fit on a single GPU even with PEFT methods (Section 6) — large multi-billion-parameter LLMs being further pre-trained or fully fine-tuned — model parallelism strategies become necessary. Fully Sharded Data Parallel (FSDP) shards a model's parameters, gradients, and optimiser states across multiple GPUs, with each GPU holding only a fraction of the full model at any time and communicating the missing pieces on demand during the forward and backward passes. DeepSpeed's ZeRO optimiser offers a similar family of sharding strategies (ZeRO stages 1, 2, and 3, corresponding to progressively more aggressive sharding of optimiser states, gradients, and parameters respectively), and integrates directly with the Hugging Face `Trainer` via a configuration file passed through `TrainingArguments`.

```
# Launching distributed training with the Trainer API (DDP, 4 GPUs)
# torchrun --nproc_per_node=4 train.py

# Enabling DeepSpeed ZeRO Stage 2 via TrainingArguments
training_args = TrainingArguments(
    output_dir='./results',
    deepspeed='ds_config_zero2.json', # path to DeepSpeed config file
    per_device_train_batch_size=8,
    gradient_accumulation_steps=4,    # effective batch size = 8 * 4 * num_gpus
    fp16=True,
)
```

A practical rule of thumb for the Gudskey team: DDP alone is sufficient and simplest for any model that fits in a single GPU's memory with room to spare for optimiser states and activations — which covers the great majority of BERT-family fine-tuning workflows. FSDP or DeepSpeed ZeRO becomes necessary only when fine-tuning models in the billions-of-parameters range without PEFT, or when memory is constrained enough that even a single replica of the model plus its optimiser state does not fit on the available hardware.

Reproducibility Checklist for Fine-Tuning Experiments

Fine-tuning runs are sensitive to several sources of randomness that can cause meaningfully different results across runs with identical hyperparameters, which matters for any research team comparing experiments across time or across team members. Before reporting or comparing fine-tuning results:

- Fix random seeds for Python, NumPy, PyTorch, and any data shuffling (`transformers.set_seed(42)`).
- Record the exact model checkpoint version and tokenizer version used.
- Report results averaged across multiple seeds (3–5 runs) for small datasets, where single-run variance can easily exceed the difference between two genuinely different hyperparameter settings.
- Log the full `TrainingArguments` configuration alongside any reported metric.

4 Learning Rate Scheduling

The learning rate is widely regarded as the single most important hyperparameter in fine-tuning a pre-trained language model. Because the model begins training from an already well-optimised point in weight space (Section 1.2), the learning rate dynamics during fine-tuning differ substantially from training a model from scratch — large or poorly scheduled learning rates can rapidly destroy the carefully learned pre-trained representations, a phenomenon closely related to catastrophic forgetting (Section 5). This section surveys the learning rate schedules most commonly used in transformer fine-tuning.

4.1 Warm-Up and Linear Decay

The standard Transformer fine-tuning learning rate schedule, used in the original BERT paper and adopted as the default in the Hugging Face `Trainer`, consists of two phases. During the warm-up phase, the learning rate is linearly increased from 0 to a peak value (typically in the range $2e-5$ to $5e-5$ for full fine-tuning) over the first 6–10% of total training steps. During the decay phase, the learning rate is linearly decreased from its peak value back to 0 over the remaining training steps.

The purpose of warm-up is to prevent large, destabilising weight updates early in training. At the very start of fine-tuning, the task-specific head is randomly initialised and produces essentially random gradients with respect to the pre-trained backbone. If the full learning rate were applied immediately, these large, noisy early gradients could propagate through the network and significantly disturb the carefully learned pre-trained weights before the task head has had a chance to stabilise. By starting at a learning rate of 0 and ramping up gradually, warm-up allows the task head to begin learning useful gradients before the backbone is exposed to its full magnitude.

```
from transformers import get_linear_schedule_with_warmup
import torch

optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5)

num_training_steps = num_epochs * len(train_dataloader)
num_warmup_steps = int(0.1 * num_training_steps) # 10% warmup

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=num_warmup_steps,
    num_training_steps=num_training_steps,
)

# Inside training loop, after optimizer.step():
scheduler.step()
```

The decay phase serves a complementary purpose: as training progresses and the model approaches a good solution for the downstream task, progressively smaller learning rates allow finer adjustments, reducing the risk of overshooting a good minimum and improving the stability of final convergence. Linear decay to exactly 0 by the final training step ensures that the very last gradient updates are vanishingly small, locking in the model's final state.

4.2 Cosine Annealing and the One-Cycle Policy

Cosine Annealing

Cosine annealing is an alternative decay schedule that follows a cosine curve rather than a straight line. After an initial warm-up phase identical to the linear schedule, the learning rate follows the function: $lr(t) = lr_peak \times 0.5 \times (1 + \cos(\pi \times t / T))$, where t is the current training step and T is the total number of decay steps. This produces a schedule that decreases slowly at first, accelerates through the middle of training, and then decreases slowly again as it approaches 0.

The practical motivation for cosine annealing is that the slow initial decrease keeps the learning rate near its peak for longer than linear decay, allowing the model more time to explore at a high learning rate before committing to fine-grained convergence. The slow final decrease, similar to linear decay, allows for stable convergence. Empirically, cosine annealing often produces marginally better final performance than linear decay, particularly for longer training runs, though the difference is typically small for standard fine-tuning regimes (2-4 epochs).

```
from transformers import get_cosine_schedule_with_warmup

scheduler = get_cosine_schedule_with_warmup(
    optimizer,
    num_warmup_steps=num_warmup_steps,
    num_training_steps=num_training_steps,
    num_cycles=0.5, # 0.5 = standard single cosine decay to 0
)
```

The One-Cycle Policy

The One-Cycle policy (Smith & Topin, 2018), originally developed for training vision models from scratch, has also been adopted in some NLP fine-tuning workflows. It combines a warm-up and decay structure with an additional momentum-scheduling component: the learning rate increases from a low value to a peak (over roughly 30–45% of training), then decreases back down to a value substantially below the initial learning rate (often by one or two orders of magnitude) for the remainder of training — a much more aggressive final decay than standard linear or cosine schedules.

Simultaneously, the optimiser's momentum (or, for Adam-family optimisers, the β_1 parameter) is scheduled inversely to the learning rate: momentum is decreased while the learning rate increases, and increased while the learning rate decreases. The intuition is that high learning rates benefit from lower momentum (to avoid overshooting), while low learning rates benefit from higher momentum (to maintain steady progress in a consistent direction). Smith & Topin's original work demonstrated that this policy enables 'super-convergence' — reaching strong performance in far fewer total training steps than standard schedules — though its benefit for transformer fine-tuning specifically is less pronounced than for training vision models from scratch, since fine-tuning already starts from a well-optimised point.

Schedule	Shape	Typical Peak LR	Best For
Linear Warmup + Decay	Linear up, linear down	2e-5 – 5e-5	Standard BERT/RoBERTa fine-tuning (default)
Cosine Annealing	Linear up, cosine down	2e-5 – 5e-5	Longer fine-tuning runs; marginal gains over linear

One-Cycle Policy	Triangular LR + inverse momentum	Higher; tuned per run	Training from scratch; less common for LLM fine-tuning
Constant with Warmup	Linear up, flat	1e-5 – 3e-5	Continued pre-training / domain adaptation (Section 7)

4.3 Differential Learning Rates Across Layers

Beyond scheduling the learning rate over time, an orthogonal technique — differential (or layer-wise) learning rates — varies the learning rate across the depth of the network at any given point in training. The motivating intuition, consistent with the probing findings discussed in Day 19 Section 9 and the partial fine-tuning rationale in Section 2.3 of this document, is that lower transformer layers encode general-purpose syntactic structure that should change only slightly during fine-tuning, while upper layers encode task-relevant semantic information that may benefit from larger updates.

In practice, this is implemented by assigning a smaller learning rate to lower layers and progressively larger learning rates to upper layers — a technique sometimes called layer-wise learning rate decay (LLRD). A common parametrisation multiplies the base learning rate by a decay factor raised to the power of the layer's depth from the output: $lr_layer_i = lr_base \times decay_factor^{(L - i)}$, where L is the total number of layers and $decay_factor$ is typically set in the range 0.9–0.95.

```
def get_layerwise_lr_params(model, base_lr=2e-5, decay_factor=0.9):
    params = []
    num_layers = model.config.num_hidden_layers
    for i, layer in enumerate(model.bert.encoder.layer):
        layer_lr = base_lr * (decay_factor ** (num_layers - i))
        params.append({'params': layer.parameters(), 'lr': layer_lr})
    # Embeddings get the smallest learning rate of all
    params.append({'params': model.bert.embeddings.parameters(),
                    'lr': base_lr * (decay_factor ** (num_layers + 1))})
    # Task head gets the full base learning rate
    params.append({'params': model.classifier.parameters(), 'lr': base_lr})
    optimizer = torch.optim.AdamW(params)
    return optimizer
```

LLRD has been shown to provide modest but consistent improvements over a single global learning rate across several GLUE tasks, and is a standard technique used in many top-performing GLUE and SuperGLUE leaderboard submissions. It is a comparatively low-cost technique to implement — requiring no architectural changes, only a different parameter grouping when constructing the optimiser — making it a reasonable default to consider whenever a single fine-tuning run is expected to be replicated across many tasks or datasets in a research pipeline.

5 Catastrophic Forgetting and Mitigation

Catastrophic forgetting refers to the tendency of a neural network to abruptly and severely lose previously learned knowledge when trained on new data or a new task. The phenomenon was first documented in the context of connectionist networks by McCloskey & Cohen (1989) and remains a central challenge in continual learning and transfer learning more broadly. In the context of fine-tuning pre-trained language models, catastrophic forgetting manifests as a degradation of the general linguistic capabilities encoded during pre-training as the model is adapted to a narrow downstream task.

5.1 Why Catastrophic Forgetting Happens

At the heart of catastrophic forgetting lies a basic property of gradient-based optimisation: when a network's parameters are updated to minimise loss on a new task or dataset, gradient descent has no built-in mechanism to protect parameter values that were important for a previous task. The same weight matrices that encoded useful pre-trained linguistic structure can be overwritten by gradient updates that are exclusively informed by the new fine-tuning objective, with no signal indicating which weights are 'important to preserve.'

This problem is compounded by overparameterisation interacting with limited fine-tuning data: a large pre-trained model like BERT-Large has 340 million parameters, while a typical fine-tuning dataset might contain only a few thousand labelled examples. The optimisation landscape is heavily underdetermined by the fine-tuning data alone, meaning many different parameter configurations could achieve low training loss on the fine-tuning task — including configurations that happen to severely disrupt the pre-trained representations in directions the fine-tuning data simply does not probe.

Catastrophic forgetting is particularly consequential in two scenarios relevant to the Gudsky team's research workflow: sequential fine-tuning on multiple tasks (where fine-tuning on Task B causes the model to forget what it learned fine-tuning on Task A), and aggressive single-task fine-tuning with too high a learning rate or too many epochs, which can erode general language understanding capabilities even while improving performance on the narrow target task, hurting the model's robustness to out-of-distribution inputs at test time.

A concrete illustration: if a BERT model is first fine-tuned on a sentiment classification dataset and subsequently fine-tuned a second time on a named entity recognition dataset, without any of the mitigation strategies discussed below, its sentiment classification accuracy on the original task will typically degrade substantially — in some documented cases by 10–20 accuracy points — even though no example from the sentiment task was ever revisited during the second fine-tuning run. The model has not become generically worse at language understanding; rather, the specific weight configuration that was well-suited to sentiment classification has been overwritten by gradient updates serving an entirely different objective.

5.2 Mitigation Strategies

Small Learning Rates and Limited Epochs

The simplest and most widely used mitigation is also the most direct: use a small learning rate (2e-5 to 5e-5, as discussed in Section 4) and train for relatively few epochs (2–4). Because the loss landscape near the pre-trained initialisation is assumed to be reasonably well-behaved, small steps are sufficient to adapt to the new task without travelling far enough to destroy useful pre-trained structure. This is why virtually every BERT fine-tuning recipe specifies such conservative learning rates compared to training from scratch (which often uses learning rates of 1e-3 or higher).

Layer Freezing

As discussed in Section 2.3, freezing some or all of the lower transformer layers during fine-tuning directly prevents forgetting in those layers, since their weights are never updated. This is a coarse but effective mitigation, particularly useful when the fine-tuning dataset is small and the risk of overfitting (which exacerbates forgetting of general structure) is high.

Elastic Weight Consolidation (EWC)

Elastic Weight Consolidation (Kirkpatrick et al., 2017) is a more principled mitigation technique originally developed for continual learning. EWC adds a regularisation term to the fine-tuning loss that penalises changes to parameters that were important for the original (pre-training) task, where importance is estimated using the diagonal of the Fisher information matrix — a measure of how sensitive the pre-training loss is to changes in each individual parameter.

The EWC loss takes the form: $L(\theta) = L_{\text{task}}(\theta) + (\lambda/2) \sum_i F_i (\theta_i - \theta_i^*)^2$, where θ_i^* are the pre-trained parameter values, F_i is the Fisher information for parameter i , and λ controls the strength of the regularisation. Parameters with high Fisher information (i.e., parameters the pre-trained model is highly sensitive to) are penalised more heavily for moving away from their pre-trained values, while parameters with low Fisher information are allowed to change more freely to fit the new task.

```
# Simplified EWC regularisation term (conceptual)
def ewc_loss(model, pretrained_params, fisher_diag, lam=1000):
    penalty = 0.0
    for name, param in model.named_parameters():
        penalty += (fisher_diag[name] * (param - pretrained_params[name]) ** 2).sum()
    return (lam / 2) * penalty

# Combined with task loss during training:
loss = task_loss(outputs, labels) + ewc_loss(model, theta_star, fisher_diag)
loss.backward()
```

Rehearsal and Replay

Rehearsal-based methods mitigate forgetting by interleaving a small sample of data representative of the original pre-training distribution (or a previous task, in the sequential fine-tuning setting) alongside the new fine-tuning data during training. This directly exposes the model to a reminder of the original distribution during every training step, counteracting the gradient pressure to forget it. In practice, this can be as simple as mixing a small percentage of generic Wikipedia/BooksCorpus-style text (with a self-supervised MLM loss) into the fine-tuning batches alongside the task-specific labelled examples.

Mixout

Mixout (Lee et al., 2020) is a regularisation technique inspired by Dropout, specifically designed to mitigate forgetting during fine-tuning. Instead of randomly zeroing out activations (as in standard Dropout), Mixout randomly replaces each parameter, with some probability p , with its corresponding pre-trained value at every training step. This stochastically anchors the model toward its pre-trained initialisation throughout training, providing a soft, randomised form of the same protective effect EWC achieves through an explicit penalty term — without needing to compute and store Fisher information estimates.

Mitigation Technique	Mechanism	Computational Overhead	Typical Use Case
----------------------	-----------	------------------------	------------------

Small LR + Few Epochs	Limit total parameter movement	None	Default for nearly all fine-tuning
Layer Freezing	Prevent updates to selected layers	Reduces overhead (fewer gradients)	Small datasets, memory-constrained settings
Elastic Weight Consolidation	Penalise change to important parameters	Moderate (Fisher info computation)	Sequential multi-task fine-tuning
Rehearsal / Replay	Mix in original-distribution data	Moderate (extra forward passes)	Continual learning, multi-domain pipelines
Mixout	Stochastic reversion to pre-trained weights	Low	General-purpose forgetting mitigation

5.3 Measuring Forgetting: Evaluation Protocols

Detecting catastrophic forgetting requires deliberately evaluating a fine-tuned model on data and tasks beyond the one it was fine-tuned for — a step that is easy to omit from a research pipeline focused narrowly on a single target metric, but essential for understanding whether a fine-tuning run has degraded the model's general capabilities. Two complementary evaluation protocols are commonly used.

- **Probing-task evaluation:** Re-run a subset of the probing classifier experiments discussed in Day 19, Section 9 (syntactic structure recovery, coreference, semantic role labelling) on the fine-tuned model's representations, and compare against the same probes run on the original pre-trained checkpoint. A significant drop in probing accuracy indicates that fine-tuning has eroded general linguistic structure, even if the target task's metric has improved.
- **Held-out task evaluation:** Evaluate the fine-tuned model on a small held-out benchmark of tasks unrelated to the fine-tuning target (a few GLUE tasks, for instance, when fine-tuning for a domain-specific application), tracking performance before and after fine-tuning. This directly measures whether general NLU capability has been preserved, independent of the probing-based syntactic and semantic measures.

In Gudsky's research context, where a single base model is sometimes adapted across multiple downstream projects over time, it is good practice to retain a small, fixed evaluation suite covering both syntactic probing and a handful of GLUE-style tasks, and to run this suite after every major fine-tuning experiment — providing an early warning signal if a particular fine-tuning configuration is unusually destructive to the model's general capabilities, before that configuration is adopted as a standard recipe across the team.

6 Parameter-Efficient Fine-Tuning (PEFT)

As pre-trained language models have grown from BERT's 110–340 million parameters to modern LLMs with tens or hundreds of billions of parameters, full fine-tuning (Section 2.2) has become prohibitively expensive for most research groups and organisations. Storing optimiser states for every parameter of a 7-billion-parameter model alone requires on the order of 50+ GB of GPU memory using standard Adam, before accounting for activations and gradients — far beyond the capacity of consumer or even many enterprise GPUs.

Parameter-Efficient Fine-Tuning (PEFT) methods address this challenge by freezing the vast majority of the pre-trained model's parameters and introducing a small number of new, trainable parameters that are sufficient to adapt the model's behaviour to a downstream task. This section covers the three most influential families of PEFT methods: adapter layers, LoRA, and prefix/prompt tuning.

6.1 Adapter Layers

Adapter layers (Houlsby et al., 2019) are small, trainable neural network modules inserted between the existing layers of a frozen pre-trained transformer. Rather than updating any of the original transformer weights, training is restricted entirely to these newly inserted adapter modules — typically less than 5% of the total model's parameter count.

Each adapter module follows a bottleneck architecture: a down-projection layer reduces the hidden dimension H (e.g., 768) to a much smaller bottleneck dimension m (e.g., 64), a nonlinearity (typically GeLU) is applied, and an up-projection layer restores the dimension back to H . A residual (skip) connection adds the adapter's output back to its input, ensuring that the adapter can default to an identity function (no transformation) at initialisation, which keeps the model's initial behaviour close to the original pre-trained model and stabilises early training.

```
import torch.nn as nn

class AdapterLayer(nn.Module):
    def __init__(self, hidden_dim=768, bottleneck_dim=64):
        super().__init__()
        self.down_project = nn.Linear(hidden_dim, bottleneck_dim)
        self.activation = nn.GELU()
        self.up_project = nn.Linear(bottleneck_dim, hidden_dim)
        # Initialise up_project near zero so adapter starts as ~identity
        nn.init.normal_(self.up_project.weight, std=1e-3)

    def forward(self, x):
        residual = x
        x = self.down_project(x)
        x = self.activation(x)
        x = self.up_project(x)
        return x + residual  # skip connection
```

Adapters are typically inserted twice per transformer layer — once after the multi-head self-attention sub-layer, and once after the feed-forward sub-layer — interleaved with the existing layer normalisation operations. Houlsby et al. demonstrated that adapter-based fine-tuning matches full fine-tuning performance on the GLUE benchmark while training only 3.6% of the parameters typically required, with negligible inference latency overhead due to the small bottleneck dimension.

6.2 LoRA: Low-Rank Adaptation

LoRA (Low-Rank Adaptation, Hu et al., Microsoft, 2021) takes a different architectural approach from adapter layers, motivated by the observation that the weight updates required to adapt a pre-trained model to a new task tend to have a low intrinsic rank — meaning the change to any given weight matrix, while potentially large in absolute terms, can be well-approximated by a low-rank decomposition.

Concretely, for a pre-trained weight matrix $W_0 \in \mathbb{R}^{(d \times k)}$, LoRA freezes W_0 entirely and represents the fine-tuning update as a product of two much smaller matrices: $W = W_0 + \alpha BA$, where $B \in \mathbb{R}^{(d \times r)}$, $A \in \mathbb{R}^{(r \times k)}$, and the rank r is chosen to be much smaller than $\min(d, k)$ — typically $r = 4, 8$, or 16 . The scalar α is a scaling factor controlling the magnitude of the adaptation's contribution relative to the frozen pre-trained weights.

During training, only A and B are updated via gradient descent; W_0 remains entirely frozen. At initialisation, A is typically initialised with small random Gaussian values and B is initialised to all zeros, ensuring that $BA = 0$ at the start of training — so the model's initial behaviour exactly matches the original pre-trained model, just as with adapter layers' zero-initialised up-projection.

```
import torch.nn as nn

class LoRALinear(nn.Module):
    def __init__(self, base_linear: nn.Linear, rank=8, alpha=16):
        super().__init__()
        self.base_linear = base_linear  # frozen pre-trained weight W0
        for p in self.base_linear.parameters():
            p.requires_grad = False
        d_in, d_out = base_linear.in_features, base_linear.out_features
        self.lora_A = nn.Parameter(torch.randn(rank, d_in) * 0.01)
        self.lora_B = nn.Parameter(torch.zeros(d_out, rank))
        self.scaling = alpha / rank

    def forward(self, x):
        base_out = self.base_linear(x)
        lora_out = (x @ self.lora_A.T) @ self.lora_B.T
        return base_out + self.scaling * lora_out
```

The parameter savings from LoRA are dramatic at large scale. For a 7-billion-parameter LLM with rank $r=8$ applied to the query and value projection matrices in every attention layer, LoRA adds only approximately 4 million trainable parameters — less than 0.1% of the total model size — while Hu et al. demonstrate that this matches full fine-tuning quality on most evaluated tasks. Because W_0 remains frozen and unchanged, multiple LoRA adapters (one per task or domain) can be trained independently and swapped in and out at inference time without needing to store multiple full copies of the base model — a significant practical advantage for serving many fine-tuned variants of the same base model.

6.3 Prefix Tuning and Prompt Tuning

Prefix tuning (Li & Liang, 2021) and prompt tuning (Lester et al., 2021) take a fundamentally different approach from adapters and LoRA: rather than introducing new trainable weights inside the transformer's layers, they introduce trainable continuous vectors that are prepended to the input sequence (or to the keys and values at every attention layer), while every original model parameter remains entirely frozen.

Prefix Tuning

Prefix tuning prepends a sequence of trainable, task-specific vectors — the 'prefix' — directly to the keys and values used in the self-attention computation at every transformer layer. These prefix vectors are not real tokens and have no corresponding embedding in the vocabulary; they are free parameters, optimised directly via gradient descent on the downstream task, that the frozen attention mechanism learns to 'attend to' as if they were context. Because a separate set of prefix vectors is injected at every layer, prefix tuning has greater expressive capacity than methods that only modify the input embedding layer, allowing it to influence the computation happening deep inside the network.

Prompt Tuning

Prompt tuning is a simplification of prefix tuning: rather than injecting trainable vectors at every layer, it introduces a small number of trainable 'soft prompt' tokens only at the input embedding layer, which are then processed by the entirely frozen transformer exactly like any other input token. Lester et al. demonstrated that as model scale increases (specifically for models in the billions-of-parameters range, such as T5-XXL), prompt tuning's performance approaches that of full fine-tuning, despite training orders of magnitude fewer parameters — though at smaller model scales (BERT-Base, BERT-Large), the performance gap relative to full fine-tuning remains more pronounced.

```
import torch.nn as nn

class PromptTuningWrapper(nn.Module):
    def __init__(self, base_model, num_virtual_tokens=20, embed_dim=768):
        super().__init__()
        self.base_model = base_model  # entirely frozen
        for p in self.base_model.parameters():
            p.requires_grad = False
        # Trainable soft prompt embeddings — the only parameters updated
        self.soft_prompt = nn.Parameter(torch.randn(num_virtual_tokens, embed_dim) * 0.01)

    def forward(self, input_embeds, attention_mask):
        batch_size = input_embeds.shape[0]
        prompt = self.soft_prompt.unsqueeze(0).expand(batch_size, -1, -1)
        full_input = torch.cat([prompt, input_embeds], dim=1)
        # Attention mask must be extended to cover the prepended prompt tokens
        prompt_mask = torch.ones(batch_size, prompt.shape[1], device=attention_mask.device)
        full_mask = torch.cat([prompt_mask, attention_mask], dim=1)
        return self.base_model(inputs_embeds=full_input, attention_mask=full_mask)
```

6.4 Comparing PEFT Methods

Method	Trainable Params (7B model)	Where Inserted	Inference Overhead	Multi-Task Serving
Adapter Layers	~0.5 – 8%	Between transformer sub- layers	Small (extra forward pass)	Swap adapter modules
LoRA	~0.01 – 0.5%	Low-rank update to weight matrices	Negligible (can merge into W)	Swap A/B matrices per task

Prefix Tuning	~0.1 – 3%	Keys/values at every attention layer	Small (longer effective sequence)	Swap prefix vectors
Prompt Tuning	<0.01%	Input embedding layer only	Minimal (few extra tokens)	Swap soft prompt only
Full Fine-Tuning	100%	Every parameter	None	Requires full model copy per task

Use Case: LLM Fine-Tuning at Bloomberg

Bloomberg GPT (Wu et al., 2023) pre-trained a 50-billion-parameter model on 363 billion tokens of financial data, then fine-tuned on named entity recognition, sentiment analysis, and question-answering tasks specific to financial documents. It outperformed GPT-4 on financial NLP benchmarks despite being roughly 10× smaller, demonstrating that domain-specific pre-training combined with targeted fine-tuning can beat much larger general-purpose models on specialised tasks — directly relevant to the case for domain adaptation discussed in Section 7.

6.5 IA³: A Lighter-Weight Alternative

IA³ (Infused Adapter by Inhibiting and Amplifying Inner Activations, Liu et al., 2022) is a further simplification of the PEFT design space, learning only three small rescaling vectors per transformer layer — applied elementwise to the keys, values, and the intermediate feed-forward activations — rather than learning low-rank matrices (LoRA) or additional bottleneck modules (adapters). Because rescaling vectors have dimensionality equal to the hidden size rather than a matrix product, IA³ introduces even fewer trainable parameters than LoRA for a comparable hidden size, while the original paper reports performance competitive with, and on some few-shot benchmarks exceeding, full fine-tuning and LoRA.

6.6 Choosing a PEFT Method

With several PEFT methods now available, selecting one for a given Gudskey research project depends on the specific constraints at hand. LoRA has emerged as the most widely adopted default in both research and industry, owing to its strong empirical performance, simple implementation, and the fact that its low-rank updates can be merged directly back into the original weight matrices after training ($W_0 + \alpha BA$ collapses into a single matrix), eliminating any inference-time overhead entirely — a property that adapter layers and prefix tuning do not share, since their additional modules remain part of the forward pass at inference time.

Prompt tuning and IA³ are preferable when the absolute minimum number of trainable parameters is required — for example, when storing many task-specific adaptations of the same base model and storage cost per task is the binding constraint. Adapter layers and prefix tuning remain useful in settings where the additional modules' slightly higher expressive capacity, at the cost of a small inference overhead, is worth the trade-off, or in codebases already built around the original adapter framework predating LoRA's widespread adoption.

6.7 Worked Example: LoRA Fine-Tuning with Hugging Face PEFT

The Hugging Face `peft` library provides production-ready implementations of LoRA, prefix tuning, prompt tuning, and IA³, designed to integrate directly with the `Trainer` API covered in Section 3. The

following example fine-tunes a sequence classification model using LoRA, illustrating how compactly the technique can be applied in practice using existing infrastructure.

```
from transformers import AutoModelForSequenceClassification, AutoTokenizer, Trainer, TrainingArguments
from peft import LoraConfig, get_peft_model, TaskType

base_model = AutoModelForSequenceClassification.from_pretrained(
    'roberta-large', num_labels=2
)

lora_config = LoraConfig(
    task_type=TaskType.SEQ_CLS,
    r=8,                # rank
    lora_alpha=16,       # scaling factor (alpha / r = 2.0)
    lora_dropout=0.1,
    target_modules=['query', 'value'], # apply LoRA to attention Q and V projections
)

model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
# trainable params: 1,572,864 || all params: 355,361,794 || trainable%: 0.4426

trainer = Trainer(
    model=model,
    args=TrainingArguments(
        output_dir='./lora-results', num_train_epochs=5,
        per_device_train_batch_size=16, learning_rate=3e-4, # higher LR typical for LoRA
    ),
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
)
trainer.train()

# After training, merge LoRA weights into the base model for zero-overhead inference
merged_model = model.merge_and_unload()
```

Two details in this example are worth highlighting for a research team adopting LoRA for the first time. First, the reported trainable parameter percentage (0.44% in this example) reflects only the query and value projection matrices being adapted; the original LoRA paper found this subset sufficient for most tasks, though `target\_modules` can be extended to include key projections and feed-forward layers if additional capacity is needed. Second, LoRA fine-tuning typically uses a substantially higher learning rate (often $1e-4$ to $3e-4$) than full fine-tuning's $2e-5$ to $5e-5$ range, since only a small number of newly initialised parameters are being trained from scratch, rather than making delicate adjustments to an already well-optimised full set of pre-trained weights — the catastrophic forgetting concerns of Section 5 are correspondingly reduced, since the vast majority of the original pre-trained weights are never touched at all.

7 Domain Adaptation vs. Task Adaptation

A subtlety often overlooked in introductory treatments of fine-tuning is the distinction between domain adaptation and task adaptation — two related but conceptually distinct forms of specialisation that a pre-trained model can undergo. Understanding this distinction is essential for designing an effective fine-tuning strategy when working with text from a specialised domain (biomedical literature, legal contracts, financial filings) and a specific downstream task (classification, extraction, generation) simultaneously.

7.1 Domain Adaptation

Domain adaptation refers to continuing the original self-supervised pre-training objective (e.g., masked language modelling) on a large, unlabelled corpus of text from the target domain, without yet introducing any task-specific labels or output head. The goal is to shift the model's general language understanding toward the vocabulary, syntax, and discourse conventions characteristic of the target domain, while preserving the broad linguistic competence acquired during the original pre-training phase.

This is precisely the approach taken by the domain-specific BERT variants discussed in Day 19, Section 7: BioMedBERT continues MLM pre-training on 18 billion words of PubMed text, Legal-BERT continues pre-training on legal corpora, and FinBERT continues pre-training on financial news. Domain adaptation does not require any labelled data — only a large corpus of representative in-domain text — making it comparatively cheap to obtain the necessary training data, though the continued pre-training process itself remains computationally expensive, often requiring days of GPU/TPU time.

7.2 Task Adaptation

Task adaptation refers to the standard fine-tuning process described throughout this document: training the model (with a task-specific output head) on labelled examples for a specific downstream task, using the standard supervised fine-tuning loss (cross-entropy for classification, span-prediction loss for QA, and so on). Task adaptation does not require the training corpus to be in any particular domain — it requires labelled examples for the specific task the model is being adapted to perform.

Task adaptation can be applied directly to a general-purpose pre-trained model (e.g., fine-tuning standard BERT-Base on a sentiment classification dataset), or it can be applied after domain adaptation has already been performed, in which case the model benefits from both the domain-specific representations learned during domain adaptation and the task-specific specialisation learned during task adaptation.

7.3 Combining Both: DAPT + TAPT

Gururangan et al. (2020), in a paper titled 'Don't Stop Pretraining', systematically studied the combination of Domain-Adaptive Pre-Training (DAPT) and Task-Adaptive Pre-Training (TAPT) across eight tasks spanning four domains (biomedical, computer science, news, and reviews). Their key findings establish a clear hierarchy of effectiveness:

- **DAPT alone:** Continuing MLM pre-training on a large unlabelled domain corpus before task fine-tuning consistently improves downstream task performance relative to fine-tuning the general-purpose model directly, across all four domains studied.
- **TAPT alone:** Continuing MLM pre-training specifically on the (unlabelled) text of the task dataset itself — even without access to any domain corpus — before task fine-tuning also improves performance, and is substantially cheaper than DAPT since it requires no additional data collection beyond what is already available for the task.

- **DAPT + TAPT combined:** Applying domain-adaptive pre-training followed by task-adaptive pre-training, and only then performing standard supervised fine-tuning, achieves the best performance of all strategies tested — the gains from DAPT and TAPT are largely complementary rather than redundant.

This finding has a direct practical implication for the Gudsky team's research workflow: when fine-tuning a model for a task in a specialised domain (e.g., a financial sentiment classifier, a biomedical NER system), it is worth considering a three-stage pipeline — domain-adaptive pre-training on a broad in-domain corpus, followed by task-adaptive pre-training on the unlabelled text from the target dataset, followed by standard supervised fine-tuning — rather than jumping directly from a general-purpose pre-trained checkpoint to task fine-tuning.

Stage	Data Required	Objective	Typical Cost
Domain-Adaptive Pre-Training (DAPT)	Large unlabelled in-domain corpus	Continued MLM	High (days of compute)
Task-Adaptive Pre-Training (TAPT)	Unlabelled task-dataset text	Continued MLM	Low (small dataset, few hours)
Task Fine-Tuning (standard)	Labelled task dataset	Supervised loss (task-specific)	Low (minutes to hours)

An important practical caveat from the Gururangan et al. study: the benefits of DAPT are largest when the target domain is most dissimilar from the original pre-training corpus (Wikipedia/BooksCorpus). For domains already well-represented in standard pre-training data (general news text, for example), the marginal benefit of DAPT is smaller, since the base model already has reasonable exposure to similar vocabulary and style.

7.4 Worked Example: Adapting a General Model for a Specialised Research Domain

To illustrate how the DAPT + TAPT framework applies concretely, consider a hypothetical research project in which a general-purpose pre-trained model must be adapted to extract structured information from a specialised, low-resource document collection — a scenario structurally similar to several of Gudsky's own applied research workflows, where source documents (technical reports, regulatory filings, specialised academic literature) differ substantially from the web and encyclopaedic text used in standard pre-training corpora.

- **Step 1 — Corpus assembly:** Collect as large an unlabelled corpus as feasible from the target domain — ideally tens of thousands of documents or more, even if only a small fraction will ultimately be labelled for the downstream task.
- **Step 2 — Domain-adaptive pre-training:** Continue MLM pre-training on this domain corpus, typically for a relatively small number of epochs (1–3) relative to original pre-training, using a conservative learning rate (1e-5 to 3e-5) and a constant-with-warmup schedule (Section 4.2) to avoid destabilising the model's general linguistic competence while still shifting its vocabulary and contextual associations toward the domain.
- **Step 3 — Task-adaptive pre-training:** Continue MLM pre-training specifically on the unlabelled text of the target task's dataset (which may be a small subset of the broader domain corpus from Step 1), further sharpening the model's familiarity with the precise document style and structure it will ultimately be evaluated on.

- **Step 4 — Task fine-tuning:** Apply standard supervised fine-tuning (Sections 2–4) on the labelled portion of the task dataset, using the model produced by Steps 2–3 as the starting checkpoint rather than the original general-purpose pre-trained model.

This staged pipeline trades additional upfront compute (Steps 2–3 require continued pre-training, which is more expensive than task fine-tuning alone) for a meaningfully stronger final model when the target domain diverges substantially from general web text — exactly the situation in which the Gururangan et al. findings predict the largest gains from DAPT and TAPT. For projects with very tight compute budgets, Step 2 can be skipped in favour of TAPT alone (Step 3 directly from the general-purpose checkpoint), which still captures a meaningful fraction of the available improvement at a much lower cost.

8 Research: Instruction Tuning and RLHF

The fine-tuning techniques discussed thus far — full fine-tuning, PEFT methods, domain/task adaptation — share a common structure: a model is adapted to perform well on one specific, narrowly-defined task at a time. The emergence of large generative language models (GPT-3 and beyond) motivated a different fine-tuning paradigm: adapting a single base model to follow natural language instructions across an open-ended range of tasks, and further aligning its outputs with human preferences for helpfulness, honesty, and harmlessness. This section surveys the two techniques most responsible for this paradigm shift: instruction tuning and Reinforcement Learning from Human Feedback (RLHF).

8.1 Instruction Tuning

Instruction tuning is a fine-tuning procedure in which a pre-trained language model is trained on a large, diverse collection of tasks, each phrased as a natural language instruction paired with the desired response — for example, 'Summarise the following article: [text]' paired with a reference summary, or 'Translate this sentence to French: [text]' paired with the correct translation. Unlike standard task adaptation (Section 7.2), which specialises a model for a single task, instruction tuning trains across hundreds or thousands of distinct tasks simultaneously, with the explicit goal of teaching the model to generalise to entirely new, unseen instructions at inference time.

FLAN (Fine-tuned LAnguage Net, Wei et al., Google, 2021) was among the first large-scale demonstrations of this approach, instruction-tuning a 137-billion-parameter model on over 60 NLP datasets reformatted as natural language instructions. The key empirical finding was that instruction tuning substantially improved zero-shot performance on held-out tasks the model had never seen an instruction for during training — the model learned a general skill of 'following instructions', not merely memorising answers to the specific training tasks.

InstructGPT (Ouyang et al., OpenAI, 2022) — the direct predecessor to ChatGPT — extended this idea by instruction-tuning GPT-3 on a dataset of instructions and demonstrations written specifically by human labellers, rather than relying solely on reformatted existing NLP datasets. This produced a model dramatically better at following user intent in open-ended, conversational instructions than the base GPT-3 model, which had only ever been trained on the next-token-prediction objective over raw internet text.

8.2 Reinforcement Learning from Human Feedback (RLHF)

While instruction tuning teaches a model to follow the format and general intent of an instruction, it does not by itself teach the model fine-grained preferences about which of several valid responses humans actually prefer — for example, which of two equally instruction-compliant summaries is more concise, more accurate, or more helpful. RLHF addresses this by directly optimising the model against a learned model of human preferences, using reinforcement learning.

The standard RLHF pipeline, as used in InstructGPT and subsequent models, consists of three stages:

- **Stage 1 — Supervised Fine-Tuning (SFT):** The base pre-trained model is fine-tuned on a dataset of high-quality human-written demonstrations of desired behaviour — essentially a focused instruction-tuning step, producing the SFT model used as the starting point for the next two stages.
- **Stage 2 — Reward Model Training:** Human labellers are shown multiple model outputs for the same prompt and asked to rank them by quality (helpfulness, accuracy, safety). A separate reward model is trained on these comparison labels to predict a scalar reward score for any given (prompt, response) pair, effectively learning to approximate human judgement of response quality.

- **Stage 3 — RL Fine-Tuning with PPO:** The SFT model is further fine-tuned using Proximal Policy Optimisation (PPO), a reinforcement learning algorithm, with the reward model from Stage 2 providing the reward signal. The model generates responses, the reward model scores them, and PPO updates the policy (the language model) to increase the probability of generating high-reward responses, while a KL-divergence penalty against the original SFT model prevents the policy from drifting too far and degenerating into reward-hacking behaviour that exploits flaws in the reward model rather than genuinely improving response quality.

```
# Simplified conceptual structure of the RLHF reward model training objective
# Given a prompt x and two responses y_winner, y_loser ranked by a human labeller:

def reward_model_loss(reward_model, x, y_winner, y_loser):
    r_winner = reward_model(x, y_winner)
    r_loser = reward_model(x, y_loser)
    # Bradley-Terry preference model: probability winner is preferred
    loss = -torch.log(torch.sigmoid(r_winner - r_loser))
    return loss

# PPO objective (conceptual, simplified):
# maximise E[ reward_model(x, y) ] - beta * KL(policy || sft_reference_model)
```

RLHF's effectiveness has made it (and its variants) the dominant alignment technique behind essentially every major conversational LLM deployed since 2022, including ChatGPT, Claude, and Gemini. However, the three-stage pipeline is notoriously complex to implement correctly and computationally expensive, since it requires maintaining and sampling from multiple large models simultaneously (the policy, the reference model, and the reward model) during the RL fine-tuning stage.

8.3 Direct Preference Optimisation (DPO): A Simpler Alternative

Direct Preference Optimisation (Rafailov et al., 2023) reformulates the RLHF objective to avoid the need for an explicit reward model and the instability of reinforcement learning training entirely. DPO shows that the same Bradley-Terry preference optimisation objective used to train the reward model in Stage 2 of RLHF can be re-derived as a direct, differentiable loss function over the policy model itself, using simple supervised learning rather than reinforcement learning. This produces models with comparable alignment quality to full RLHF, while being substantially simpler to implement, more stable to train, and far less computationally expensive, since it eliminates the need for online sampling and a separate reward model during training. DPO has become a widely adopted alternative to RLHF for organisations and research groups without the infrastructure to support a full PPO-based training pipeline.

Research Spotlight: QLoRA — Quantised Low-Rank Adaptation (Dettmers et al., 2023)

QLoRA fine-tunes a 4-bit quantised base model with LoRA adapters (Section 6.2) layered on top, enabling fine-tuning of a 65-billion-parameter LLaMA model on a single 48GB A100 GPU — a task that would otherwise

require a multi-GPU cluster using standard full-precision fine-tuning.

Combined with the Hugging Face PEFT library, QLoRA democratised LLM fine-tuning, making it accessible on

consumer-grade GPUs (RTX 3090/4090) for the first time, and is now commonly used as the default approach

for fine-tuning open-weight LLMs (LLaMA, Mistral, Falcon) for both research and applied projects, including

instruction-tuning and RLHF/DPO alignment pipelines on constrained hardware budgets.

8.4 The Alignment Tax and Open Research Questions

A consistent empirical observation across RLHF research is the existence of an alignment tax: models fine-tuned heavily toward human preference alignment sometimes exhibit small regressions on certain standard NLU benchmarks (including some GLUE and SuperGLUE-style tasks discussed in Day 19, Section 8) relative to the supervised fine-tuned model that preceded the RL stage. This is generally attributed to the RL fine-tuning stage optimising specifically for the reward model's notion of a good response, which is correlated with but not identical to performance on traditional academic NLU benchmarks — a form of the same general specialisation trade-off discussed throughout this document, just at the scale of an entire alignment pipeline rather than a single task.

Mitigating the alignment tax, improving the robustness of reward models to reward hacking, and reducing the human-labelling cost of preference data collection (which scales poorly compared to the largely automatic self-supervised pre-training and instruction-tuning data pipelines) remain active areas of research as of the time of writing. Constitutional AI (Bai et al., 2022) and related approaches reduce reliance on human preference labels by having a model critique and revise its own outputs against a set of written principles, then training on the resulting self-generated preference data — an approach intended to scale alignment training without a proportional increase in human labelling effort.

For the Gudskey research team, the throughline across this entire document is worth stating explicitly: every fine-tuning technique surveyed — from the simplest feature-extraction head to the full three-stage RLHF pipeline — is fundamentally a method for deciding which knowledge in a pre-trained model to preserve, which to adjust, and by how much. Full fine-tuning grants maximum flexibility at maximum cost and maximum forgetting risk; PEFT methods constrain that flexibility deliberately, in exchange for efficiency and stability; domain and task adaptation sequence that flexibility in stages matched to the structure of the target distribution; and instruction tuning and RLHF extend the same underlying logic to the qualitatively harder problem of aligning a model's behaviour with open-ended human intent rather than a single fixed task. Recognising this shared structure is what allows a researcher to reason about a novel fine-tuning problem from first principles, rather than needing a separate playbook for every new combination of model, task, and constraint.

8.5 Summary: From Task Fine-Tuning to Alignment

Technique	Goal	Data Required	Key Reference
Standard Task Fine-Tuning	Specialise for one specific task	Labelled examples for the task	Devlin et al., 2018 (BERT)
Instruction Tuning	Generalise to follow novel instructions	Diverse instruction/response pairs	Wei et al., 2021 (FLAN)
RLHF	Align outputs with human preferences	Human preference comparisons	Ouyang et al., 2022 (InstructGPT)

DPO	Align outputs, simpler than RLHF	Human preference comparisons	Rafailov et al., 2023
QLoRA	Make any of the above feasible on limited hardware	Same as base technique + quantisation	Dettmers et al., 2023

The future of AI is bright, and you can be part of it!